

```

# =====
#             AI STUDY ASSISTANT - COMPLETE PROJECT
# =====

# ----- 1. INSTALL LIBRARIES -----

import subprocess
import sys

packages = [
    "transformers",
    "sentencepiece",
    "torch",
    "nltk",
    "scikit-learn"
]

for package in packages:
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "-q", package]
    )

import re
import random
import time
import warnings

warnings.filterwarnings("ignore")

import nltk
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

from transformers import pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# =====
#             PROJECT INFORMATION
# =====

print("=" * 70)
print("             AI STUDY ASSISTANT")
print("=" * 70)
print("VITYarthi - Build Your Own Project")
print()
print("Modules:")
print("1. Study Material Input")
print("2. Text Preprocessing")
print("3. AI Summarization")
print("4. Question Generation")
print("5. Quiz & Answer Evaluation")
print("6. AI Study Assistant / Question Answering")
print("=" * 70)

# =====
#             MODULE 1: INPUT HANDLING
# =====

def get_study_material():

    print("\n" + "=" * 70)
    print("MODULE 1 - STUDY MATERIAL INPUT")
    print("=" * 70)

    print("\nEnter or paste your study material.")
    print("Type END on a new line when you finish.\n")

    lines = []

    while True:
        line = input()

```

```

        if line.strip().upper() == "END":
            break

        lines.append(line)

    material = "\n".join(lines).strip()

    if not material:
        print("\nERROR: Study material cannot be empty.")
        return None

    if len(material) < 100:
        print("\nWARNING: The material is very short.")
        print("For better results, provide more study content.")

    print("\nStudy material received successfully.")
    print("Characters:", len(material))
    print("Words:", len(material.split()))

    return material

# =====
#             MODULE 2: TEXT PREPROCESSING
# =====

def preprocess_text(text):

    print("\n" + "=" * 70)
    print("MODULE 2 - TEXT PREPROCESSING")
    print("=" * 70)

    # Remove excessive spaces
    text = re.sub(r"\s+", " ", text)

    # Remove unnecessary special characters
    text = re.sub(r"[^\w\s.,!?:;%+~\-\_]", "", text)

    text = text.strip()

    # Sentence splitting
    sentences = nltk.sent_tokenize(text)

    print("Preprocessing completed.")
    print("Number of sentences:", len(sentences))
    print("Number of words:", len(text.split()))

    return text, sentences

# =====
#             MODULE 3: AI SUMMARIZATION
# =====

def load_summarizer():

    print("\nLoading AI summarization model...")
    print("This may take some time during the first run.\n")

    try:
        model = pipeline(
            "summarization",
            model="sshleifer/distilbart-cnn-12-6"
        )

        print("Summarization model loaded successfully.")

        return model

    except Exception as e:

        print("Could not load summarization model.")
        print("Error:", e)

        return None

```

```

def split_into_chunks(text, max_words=350):

    words = text.split()

    chunks = []

    for i in range(0, len(words), max_words):
        chunk = " ".join(words[i:i + max_words])

        if len(chunk.split()) >= 30:
            chunks.append(chunk)

    return chunks

def generate_summary(text, summarizer):

    print("\n" + "=" * 70)
    print("MODULE 3 - AI SUMMARIZATION")
    print("=" * 70)

    if summarizer is None:
        print("Summarizer unavailable.")
        return ""

    chunks = split_into_chunks(text)

    if not chunks:
        chunks = [text]

    summaries = []

    for i, chunk in enumerate(chunks):

        print(f"Processing section {i + 1}/{len(chunks)}...")

        try:

            word_count = len(chunk.split())

            max_length = min(130, max(50, word_count // 2))
            min_length = min(50, max(20, word_count // 4))

            if max_length <= min_length:
                max_length = min_length + 10

            result = summarizer(
                chunk,
                max_length=max_length,
                min_length=min_length,
                do_sample=False
            )

            summaries.append(result[0]["summary_text"])

        except Exception as e:

            print("Warning while summarizing a section:", e)

            # Fallback: first few sentences
            sentences = nltk.sent_tokenize(chunk)
            summaries.append(" ".join(sentences[:3]))

    final_summary = " ".join(summaries)

    print("\nSUMMARY")
    print("-" * 70)
    print(final_summary)

    return final_summary

# =====
#          MODULE 4: KEY POINT EXTRACTION
# =====

def extract_key_points(text, number_of_points=8):

```

```

print("\n" + "=" * 70)
print("KEY POINTS")
print("=" * 70)

sentences = nltk.sent_tokenize(text)

if len(sentences) <= number_of_points:
    selected = sentences
else:
    # TF-IDF based important sentence selection
    try:
        vectorizer = TfidfVectorizer(
            stop_words="english"
        )

        matrix = vectorizer.fit_transform(sentences)

        scores = matrix.sum(axis=1).A1

        ranked = sorted(
            enumerate(scores),
            key=lambda x: x[1],
            reverse=True
        )

        selected_indices = sorted(
            [x[0] for x in ranked[:number_of_points]]
        )

        selected = [sentences[i] for i in selected_indices]

    except Exception:
        selected = sentences[:number_of_points]

for i, point in enumerate(selected, 1):
    print(f"{i}. {point}")

return selected

# =====
#          MODULE 5: QUESTION GENERATION
# =====

def generate_questions(sentences, number_of_questions=5):

    print("\n" + "=" * 70)
    print("MODULE 5 - QUESTION GENERATION")
    print("=" * 70)

    usable_sentences = []

    for sentence in sentences:

        words = sentence.split()

        if len(words) >= 7:
            usable_sentences.append(sentence)

    if len(usable_sentences) < 3:

        print("Not enough suitable sentences for question generation.")

        return []

    # Select sentences
    selected = random.sample(
        usable_sentences,
        min(number_of_questions, len(usable_sentences))
    )

    questions = []

    for sentence in selected:

```

```

for sentence in selected:

    words = sentence.split()

    # Find useful words
    candidates = []

    for word in words:

        clean_word = re.sub(
            r"^[A-Za-z0-9]",
            "",
            word
        )

        if len(clean_word) >= 5:
            candidates.append(clean_word)

    if not candidates:
        continue

    answer = random.choice(candidates)

    # Replace selected word in sentence
    pattern = re.compile(
        re.escape(answer),
        re.IGNORECASE
    )

    question_sentence = pattern.sub(
        "_____",
        sentence,
        count=1
    )

    if question_sentence == sentence:
        continue

    question = {
        "question": "Fill in the blank: " + question_sentence,
        "answer": answer
    }

    questions.append(question)

# If not enough questions, create simple conceptual questions
if len(questions) == 0:

    for sentence in selected:

        questions.append({
            "question": "Explain the following concept:\n" + sentence,
            "answer": sentence
        })

print("Generated questions:", len(questions))

for i, q in enumerate(questions, 1):

    print(f"\nQ{i}. {q['question']}")
    print("Answer:", q["answer"])

return questions

# =====
#           MODULE 6: QUIZ SYSTEM
# =====

def run_quiz(questions):

    print("\n" + "=" * 70)
    print("MODULE 6 - QUIZ & ANSWER EVALUATION")
    print("=" * 70)

    if not questions:
        print("No questions available.")
        return 0

```

```

score = 0
total = len(questions)

print("\nAnswer each question.")
print("For Fill-in-the-Blank questions, type the missing word.")
print()

for i, q in enumerate(questions, 1):

    print("-" * 70)
    print(f"Question {i}/{total}")
    print(q["question"])

    user_answer = input("\nYour answer: ").strip()

    correct_answer = q["answer"].strip()

    # Normalize answers
    user_clean = re.sub(
        r"^[^a-zA-Z0-9 ]",
        "",
        user_answer
    ).lower()

    correct_clean = re.sub(
        r"^[^a-zA-Z0-9 ]",
        "",
        correct_answer
    ).lower()

    if user_clean == correct_clean:

        print("Correct!")
        score += 1

    elif (
        user_clean in correct_clean
        or correct_clean in user_clean
    ) and len(user_clean) > 3:

        print("Correct / Acceptable!")
        score += 1

    else:

        print("Incorrect.")
        print("Correct answer:", correct_answer)

percentage = (score / total) * 100

print("\n" + "=" * 70)
print("QUIZ RESULT")
print("=" * 70)

print("Score:", score, "/", total)
print(f"Percentage: {percentage:.2f}%")

if percentage >= 80:
    print("Performance: Excellent")

elif percentage >= 60:
    print("Performance: Good")

elif percentage >= 40:
    print("Performance: Needs Improvement")

else:
    print("Performance: More Practice Recommended")

return percentage

# =====
#     MODULE 7: AI STUDY ASSISTANT / Q&A
# =====

```

```

def answer_question(question, study_material):

    print("\n" + "=" * 70)
    print("MODULE 7 - AI STUDY ASSISTANT")
    print("=" * 70)

    if not question.strip():

        print("Please enter a question.")

        return ""

    sentences = nltk.sent_tokenize(study_material)

    if not sentences:

        return "No study material is available."

    try:

        # TF-IDF retrieval
        documents = sentences + [question]

        vectorizer = TfidfVectorizer(
            stop_words="english"
        )

        matrix = vectorizer.fit_transform(documents)

        similarities = cosine_similarity(
            matrix[-1],
            matrix[:-1]
        )[0]

        best_indices = similarities.argsort()[-3:][::-1]

        relevant_sentences = []

        for index in best_indices:

            if similarities[index] > 0:

                relevant_sentences.append(
                    sentences[index]
                )

        if not relevant_sentences:

            return (
                "I could not find a relevant answer "
                "in the provided study material."
            )

        answer = " ".join(relevant_sentences)

        print("\nQuestion:")
        print(question)

        print("\nAnswer based on your study material:")
        print("-" * 70)
        print(answer)

        return answer

    except Exception as e:

        print("Error while answering question:", e)

        return "Unable to process the question."

# =====
#             MODULE 8: STUDY REPORT
# =====

def generate_study_report(
    material,
    summary,

```

```

    key_points,
    quiz_percentage
):

    print("\n" + "=" * 70)
    print("STUDY REPORT")
    print("=" * 70)

    print("\nAI STUDY ASSISTANT REPORT")
    print("-" * 70)

    print("Total study material words:",
          len(material.split()))

    print("Summary words:",
          len(summary.split()) if summary else 0)

    print("Important points:",
          len(key_points))

    print(
        "Quiz performance:",
        f"{quiz_percentage:.2f}%"
    )

    print("\nStudy session completed successfully.")

# =====
#                               MAIN APPLICATION
# =====

def main():

    # -----
    # STEP 1: GET MATERIAL
    # -----

    material = get_study_material()

    if material is None:
        return

    # -----
    # STEP 2: PREPROCESS
    # -----

    processed_text, sentences = preprocess_text(material)

    # -----
    # STEP 3: LOAD AI MODEL
    # -----

    summarizer = load_summarizer()

    # -----
    # STEP 4: SUMMARY
    # -----

    summary = generate_summary(
        processed_text,
        summarizer
    )

    # -----
    # STEP 5: KEY POINTS
    # -----

    key_points = extract_key_points(
        processed_text,
        number_of_points=8
    )

    # -----
    # STEP 6: QUESTIONS
    # -----

```

```

questions = generate_questions(
    sentences,
    number_of_questions=5
)

# -----
# STEP 7: QUIZ
# -----

quiz_percentage = 0

if questions:

    take_quiz = input(
        "\nDo you want to take the quiz? (yes/no): "
    ).strip().lower()

    if take_quiz in ["yes", "y"]:

        quiz_percentage = run_quiz(
            questions
        )

# -----
# STEP 8: STUDY ASSISTANT
# -----

while True:

    print("\n" + "=" * 70)
    print("AI STUDY ASSISTANT - ASK A QUESTION")
    print("=" * 70)

    question = input(
        "\nEnter your question "
        "(type EXIT to finish): "
    ).strip()

    if question.upper() == "EXIT":
        break

    answer_question(
        question,
        processed_text
    )

# -----
# STEP 9: FINAL REPORT
# -----

generate_study_report(
    material,
    summary,
    key_points,
    quiz_percentage
)

print("\n" + "=" * 70)
print("PROJECT EXECUTION COMPLETED")
print("=" * 70)

print("\nThe AI Study Assistant demonstrated:")
print("✓ Study material input")
print("✓ Text preprocessing")
print("✓ AI summarization")
print("✓ Important point extraction")
print("✓ Question generation")
print("✓ Quiz and answer evaluation")
print("✓ Study material based Q&A")
print("✓ Basic performance reporting")

print("\nYou can now take screenshots of the outputs")
print("for your project report and GitHub documentation.")

main()

```



```

=====
                        AI STUDY ASSISTANT
=====
VITyarthi - Build Your Own Project

Modules:
1. Study Material Input
2. Text Preprocessing
3. AI Summarization
4. Question Generation
5. Quiz & Answer Evaluation
6. AI Study Assistant / Question Answering
=====

=====
MODULE 1 - STUDY MATERIAL INPUT
=====

Enter or paste your study material.
Type END on a new line when you finish.

Artificial Intelligence (AI) is a branch of computer science that focuses on creating machines capable of performing tasks that
end

Study material received successfully.
Characters: 671
Words: 93

=====
MODULE 2 - TEXT PREPROCESSING
=====

Preprocessing completed.
Number of sentences: 6
Number of words: 93

Loading AI summarization model...
This may take some time during the first run.

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Could not load summarization model.
Error: "Unknown task summarization, available tasks are ['any-to-any', 'audio-classification', 'automatic-speech-recognition', '

=====
MODULE 3 - AI SUMMARIZATION
=====

Summarizer unavailable.

=====
KEY POINTS
=====
1. Artificial Intelligence (AI) is a branch of computer science that focuses on creating machines capable of performing tasks th
2. These tasks include learning, reasoning, problem solving, understanding natural language, and recognizing images.
3. Machine Learning is a subset of Artificial Intelligence that enables computers to learn from data without being explicitly pr
4. Deep Learning is a further subset of Machine Learning that uses artificial neural networks with multiple layers to learn comp
5. AI is widely used in healthcare, education, finance, transportation, and entertainment.
6. END

=====
MODULE 5 - QUESTION GENERATION
=====

Generated questions: 5

Q1. Fill in the blank: Machine Learning is a subset of Artificial Intelligence that enables _____ to learn from data withou
Answer: computers

Q2. Fill in the blank: AI is widely used in healthcare, education, _____, transportation, and entertainment.
Answer: finance

Q3. Fill in the blank: These tasks _____ learning, reasoning, problem solving, understanding natural language, and recogniz
Answer: include

Q4. Fill in the blank: Deep _____ is a further subset of Machine Learning that uses artificial neural networks with multipl
Answer: Learning

Q5. Fill in the blank: Artificial Intelligence (AI) is a branch of _____ science that focuses on creating machines capable
Answer: computer

Do you want to take the quiz? (yes/no): yes

=====
MODULE 6 - QUIZ & ANSWER EVALUATION
=====

```

Answer each question.

For Fill-in-the-Blank questions, type the missing word.

Question 1/5

Fill in the blank: Machine Learning is a subset of Artificial Intelligence that enables _____ to learn from data without be

Your answer: artificial intelligence

Incorrect.

Correct answer: computers

Question 2/5

Fill in the blank: AI is widely used in healthcare, education, _____, transportation, and entertainment.

Your answer: networks

Incorrect.

Correct answer: finance

Question 3/5

Fill in the blank: These tasks _____ learning, reasoning, problem solving, understanding natural language, and recognizing

Your answer: entertainment

Incorrect.

Correct answer: include

Question 4/5

Fill in the blank: Deep _____ is a further subset of Machine Learning that uses artificial neural networks with multiple la

Your answer: ideas

Incorrect.

Correct answer: Learning

Question 5/5

Fill in the blank: Artificial Intelligence (AI) is a branch of _____ science that focuses on creating machines capable of p

Your answer: computer science